OPERATING SYSTEMS

UNIT – II: PROCESSES, THREADS & CPU SCHEDULING

SECTION B – 5-MARK QUESTIONS IN 10-MARK ANSWER FORMAT

12 Theory Answers + 4 Step-by-Step Numerical Problems

Prepared in easy English with examples, labelled diagrams, comparison tables, formulas, Gantt charts and conclusions.

Q1. Describe message-passing and shared-memory models of Inter-Process Communication (IPC).

5 Marks – Expanded 10 Marks Format

1. Introduction

Inter-Process Communication (IPC) is a method that allows two or more running processes to exchange information and coordinate their activities. A process is a program that is currently running. Example: two applications sharing information.

Shared-Memory Model

- The OS creates a common memory area for two or more processes.
- One process writes data into the shared area and another process reads it.
- Processes must coordinate their access to avoid conflicts.
- Shared memory is usually fast for large amounts of data.

Shared-Memory Diagram

diagram

Message-Passing Model

- Processes communicate by sending and receiving messages.
- They do not need to share the same memory area.
- The OS helps transfer messages between processes.
- It is useful on the same computer and across networks.

Message-Passing Diagram

diagram

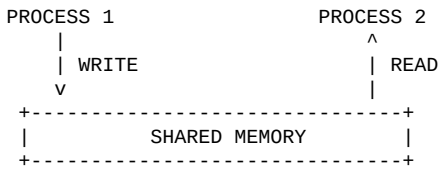
Differences

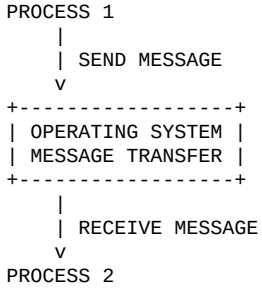
table

Conclusion

IPC allows processes to exchange information. Shared memory uses a common memory area, whereas message passing uses sending and receiving messages.

Remember: Shared Memory = Common Memory; Message Passing = Send and Receive Messages.





Shared Memory	Message Passing
Uses a common memory area.	Uses messages to communicate.
Processes access the same memory.	Processes send and receive messages.
Usually fast for large data transfers.	May have communication overhead.
Requires synchronization.	Communication is coordinated through message operations.

Shared Memory	Message Passing
Commonly used on one computer.	Useful on one computer and across networks.

Q2. Discuss multithreading models in detail with neat diagrams.

5 Marks – Expanded 10 Marks Format

1. Introduction

A thread is the smallest unit of CPU execution within a process. Multithreading means running multiple threads within the same process. Example: a browser can load a webpage while another thread handles user input.

Many-to-One Model

- Many user-level threads are connected to one kernel thread.
- Only one thread can execute at a time through that kernel thread.
- If one thread blocks, all threads may be blocked.
- It does not provide true parallel execution on multiple CPU cores.

Diagram – Many-to-One

diagram

One-to-One Model

- Each user thread is connected to a separate kernel thread.
- Threads can execute independently and may run in parallel on multiple CPU cores.
- Creating many kernel threads can increase overhead.
- Modern Windows systems use one-to-one mapping.

Diagram – One-to-One

diagram

Many-to-Many Model

- Many user threads are mapped to multiple kernel threads.
- The number of kernel threads may be equal to or smaller than the number of user threads.
- It provides flexibility and concurrency.
- It can reduce the need to create one kernel thread for every user thread.

Diagram – Many-to-Many

diagram

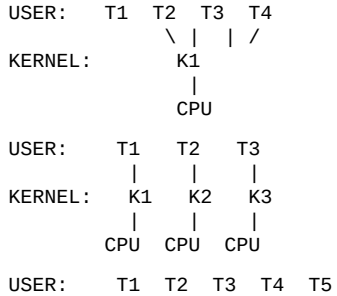
Comparison

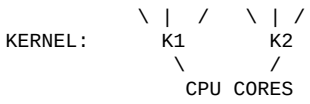
table

Conclusion

Multithreading allows multiple threads to work inside one process. The three models describe how user threads are connected to kernel threads.

Remember: Many-to-One = Many to One; One-to-One = Each to Each; Many-to-Many = Many to Multiple.





Many-to-One	One-to-One	Many-to-Many
Many user threads map to one kernel thread.	Each user thread maps to one kernel thread.	Many user threads map to multiple kernel threads.
Limited concurrency.	Supports concurrency and parallelism.	Supports concurrency and flexible mapping.
One blocking thread may block all.	Blocking one thread normally does not block others.	Blocking one thread does not normally block all.
Low kernel-thread overhead.	Higher kernel-thread overhead.	Flexible balance of user and kernel threads.

Q3. Discuss Inter-Process Communication (IPC) and its primary mechanisms.

5 Marks – Expanded 10 Marks Format

1. Introduction

IPC allows two or more processes to exchange information and coordinate their activities. It helps processes work together. Example: a client application sends a request to a server.

Shared Memory

- Multiple processes access the same memory area.
- One process writes data and another reads it.
- Synchronization is needed to avoid conflicts.

Message Passing

- Processes send and receive messages.
- The OS helps transfer the messages.
- No common memory area is required.

Other IPC Mechanisms

- Pipes: provide a channel for one process to send data to another; ordinary pipes commonly connect related processes.
- Message Queues: store messages temporarily until another process reads them.
- Sockets: allow processes to communicate on the same computer or across a network.
- Signals: notify a process that an event has occurred; they usually carry little or no data.

IPC Mechanisms Diagram

diagram

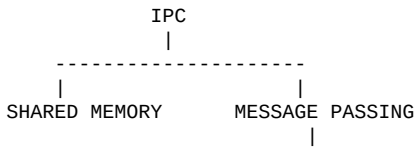
Advantages

- Allows processes to share information.
- Improves cooperation and multitasking.
- Helps applications communicate.
- Supports communication across networks.

Conclusion

IPC is an important OS feature. Processes communicate using shared memory, message passing, pipes, message queues, sockets and signals.

Remember: IPC = Communication Between Processes.



```
 --------------------
 |       |          |
PIPE  MESSAGE    SOCKETS
      QUEUE
 SIGNALS
```

Q4. Describe the role of a dispatcher in CPU scheduling and explain dispatch latency.

5 Marks – Expanded 10 Marks Format

1. Introduction

CPU Scheduling selects a process from the ready queue and gives it the CPU. The dispatcher transfers CPU control to the process selected by the scheduler. Example: a teacher gives the next student a turn.

Role of Dispatcher

- Context Switching: saves the current process state and restores the selected process state.
- Switching to User Mode: changes the CPU to user mode when required.
- Starting Execution: transfers CPU control to the selected process so it can start or resume.

Dispatcher Diagram

diagram

Dispatch Latency

Dispatch latency is the time taken to stop one process and start another. For example, if switching from P1 to P2 takes 2 milliseconds, dispatch latency is 2 milliseconds.

Factors Affecting Dispatch Latency

- Saving the current process state.
- Restoring the next process state.
- Switching between operating modes.
- Memory-related operations and OS overhead.

Importance

- Supports CPU scheduling and multitasking.
- Transfers CPU control correctly.
- Allows processes to resume execution.

Conclusion

The dispatcher transfers CPU control to the process selected by the CPU scheduler. Dispatch latency is the time required to complete this transfer.

Remember: Dispatcher = Gives CPU to selected process; Dispatch Latency = Time taken to switch CPU control.

```
READY QUEUE
[P1] [P2] [P3]
      |
      v
CPU SCHEDULER
      |
      v
SELECT P1
      |
      v
DISPATCHER
      |
      v
     CPU

PROCESS P1 RUNNING
        |
        v
   SAVE P1 STATE
        |
        v
```

```
   RESTORE P2 STATE
         |
         v
PROCESS P2 RUNNING

   <--- DISPATCH LATENCY --->
```

Q5. Explain FCFS scheduling with an example and discuss the Convoy Effect.

5 Marks – Expanded 10 Marks Format

1. Introduction

First-Come, First-Served (FCFS) is a CPU scheduling algorithm. The process that arrives first gets the CPU first. It follows FIFO (First-In, First-Out).

Working

- Processes enter the ready queue in arrival order.
- The first process gets the CPU.
- It runs until it finishes or blocks.
- Then the next process gets the CPU.
- FCFS is non-preemptive.

Example Data

P1: AT=0, BT=5; P2: AT=1, BT=3; P3: AT=2, BT=2.

Gantt Chart

diagram

Calculations

- Completion Time (CT): P1=5, P2=8, P3=10.
- Turnaround Time: TAT = CT – AT. P1=5–0=5; P2=8–1=7; P3=10–2=8.
- Waiting Time: WT = TAT – BT. P1=5–5=0; P2=7–3=4; P3=8–2=6.
- Average WT = (0+4+6)/3 = 3.33 time units.
- Average TAT = (5+7+8)/3 = 6.67 time units.

Convoy Effect

The Convoy Effect occurs when a long process occupies the CPU and many short processes wait behind it. Example: P1 needs 20 ms, P2 needs 2 ms and P3 needs 3 ms. If P1 arrives first, P2 and P3 must wait.

Convoy Diagram

diagram

Advantages and Disadvantages

- Advantages: simple, easy to implement and no starvation due to priority selection.
- Disadvantages: may cause the Convoy Effect, high average waiting time and poor response for interactive systems.

Conclusion

FCFS executes processes in arrival order. It is simple, but a long process can make shorter processes wait.

Remember: FCFS = First Arrival, First Execution.

```
   P1       P2      P3
|--------|------|----|
0        5      8   10

   P1 (20 ms)       P2 (2) P3 (3)
|-------------------|------|------|
0                  20     22     25
       P2 and P3 wait behind P1
```

Q6. Differentiate between preemptive and non-preemptive scheduling with suitable examples.

5 Marks – Expanded 10 Marks Format

1. Introduction

CPU scheduling decides which process gets the CPU. Preemptive and non-preemptive scheduling differ in whether the OS can take the CPU away from a running process.

Preemptive Scheduling

- The OS can interrupt a running process and give the CPU to another process.
- A higher-priority or newly scheduled process may get the CPU.
- The interrupted process waits and can continue later.
- Example: Round Robin.

Preemptive Diagram

diagram

Non-Preemptive Scheduling

- The CPU is not taken away while a process is running.
- The process keeps the CPU until it finishes or blocks.
- Other processes wait in the ready queue.
- Example: FCFS.

Non-Preemptive Diagram

diagram

Differences

table

Advantages and Disadvantages

- Preemptive: better response for interactive systems, but more context-switching overhead and more complex management.
- Non-preemptive: simple and usually fewer context switches, but long processes may delay short processes.

Conclusion

Preemptive scheduling allows the OS to interrupt a running process. Non-preemptive scheduling lets the process continue until it finishes or blocks.

Remember: Preemptive = CPU can be taken; Non-preemptive = Process keeps CPU until it releases it.

```
PROCESS P1 RUNNING
       |
       v
NEW PROCESS ARRIVES
       |
       v
CPU INTERRUPTED
       |
       v
PROCESS P2 RUNNING
```

```
PROCESS P1 RUNNING
       |
       v
P1 COMPLETES / BLOCKS
       |
       v
PROCESS P2 RUNNING
```

Preemptive	Non-Preemptive
CPU can be taken from a process.	CPU is not taken while process is running.
Processes can be interrupted.	Process runs until it finishes or blocks.

Preemptive	Non-Preemptive
More responsive for interactive systems.	May cause longer waits.
Usually more context switching.	Usually fewer context switches.
Example: Round Robin.	Example: FCFS.

Q7. Write short notes on Thread Libraries (POSIX Pthreads, Win32 and Java Threads).

5 Marks – Expanded 10 Marks Format

1. Introduction

A Thread Library is a collection of functions or methods that helps programmers create, manage and control threads in a program.

POSIX Pthreads

- Used in UNIX-like systems.
- Creates, manages and synchronizes threads.
- Important functions: pthread_create() creates a thread; pthread_join() waits for it; pthread_exit() terminates it; pthread_mutex_lock() locks a mutex.

Pthreads Diagram

diagram

Win32 Threads

- Thread API provided by Microsoft Windows.
- Important functions: CreateThread() creates a thread; WaitForSingleObject() waits for an object or thread; ExitThread() terminates the calling thread; CloseHandle() closes a handle.

Java Threads

- Java provides built-in multithreading support.
- Threads can be created using the Thread class or Runnable interface.
- Important methods: start(), run(), sleep() and join().

Comparison

table

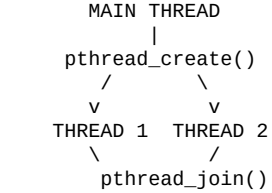
Advantages

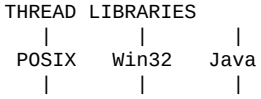
- Make thread creation easier.
- Support multitasking and application responsiveness.
- Help programmers manage and synchronize threads.

Conclusion

POSIX Pthreads is commonly used in UNIX-like systems, Win32 Threads in Windows and Java Threads in Java applications.

Remember: POSIX = UNIX-like systems; Win32 = Windows; Java = Java applications.





```
UNIX   WINDOWS  JAVA
```

POSIX Pthreads	Win32 Threads	Java Threads
UNIX-like systems.	Windows.	Java applications.
C/POSIX API.	Windows API.	Java language support.
pthread_create().	CreateThread().	start().
POSIX thread functions.	Windows thread functions.	Thread class and Runnable interface.

Q8. Explain the process concept with a neat diagram of process states.

5 Marks – Expanded 10 Marks Format

1. Introduction

A process is a program that is currently running. A program is a set of instructions stored on a computer. When it starts executing, it becomes a process. Example: opening Chrome creates a running process.

Parts of a Process

- Program Code: instructions of the program.
- Data: information used by the program.
- Stack: temporary data such as function calls and local variables.
- Heap: dynamically allocated memory.
- Program Counter: address of the next instruction.

Five Process States

- New: process is being created.
- Ready: process is ready and waiting for CPU time.
- Running: process is currently using the CPU.
- Waiting/Blocked: process is waiting for an event or I/O.
- Terminated: process has completed and its resources can be released.

Process State Diagram

diagram

State Transitions

- New → Ready: OS admits the process.
- Ready → Running: CPU is assigned.
- Running → Waiting: process requests I/O or waits for an event.
- Waiting → Ready: event or I/O completes.
- Running → Ready: CPU is taken away during preemptive scheduling.
- Running → Terminated: process finishes.

Advantages of Process Management

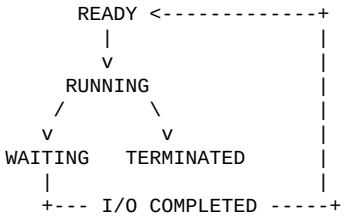
- Allows multiple programs to run.
- Manages CPU resources and supports multitasking.
- Tracks process execution and state.

Conclusion

A process is a program in execution. It moves through New, Ready, Running, Waiting and Terminated states.

Remember: New → Ready → Running → Waiting → Ready → Terminated.

```
          NEW
           |
           v
```



Q9. What is a Process Control Block (PCB)? Explain its components.

5 Marks – Expanded 10 Marks Format

1. Introduction

A Process Control Block (PCB) is a data structure maintained by the OS to store information about a process. Every process has its own PCB. It is like a student record that stores important details.

Components of PCB

- Process ID (PID): unique number used to identify a process.
- Process State: current state such as New, Ready, Running, Waiting or Terminated.
- Program Counter: address of the next instruction.
- CPU Registers: temporary CPU values saved when a process is switched out.
- CPU Scheduling Information: priority and queue-related details.
- Memory Management Information: information such as page tables and memory limits.
- Accounting Information: resource usage such as CPU time.
- I/O Status Information: allocated devices and open files.

PCB Diagram

diagram

Importance

- Identifies processes.
- Stores information needed for process management.
- Supports context switching and CPU scheduling.
- Helps manage memory and I/O resources.

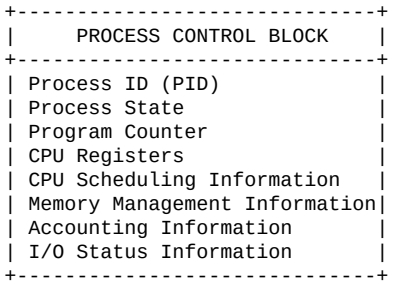
PCB During Context Switching

- Save the current process information in its PCB.
- Select another process.
- Restore that process information from its PCB.
- Continue its execution.

Conclusion

PCB is an important data structure. It stores PID, state, program counter, registers, scheduling, memory, accounting and I/O information.

Remember: PCB = Complete Information About a Process.



Q10. Explain process schedulers: Long-term, Short-term and Medium-term.

5 Marks – Expanded 10 Marks Format

1. Introduction

A Process Scheduler selects processes and decides when they should enter memory or get CPU time. The main purpose is to manage processes efficiently.

Long-term Scheduler (Job Scheduler)

- Selects processes from the job pool.
- Loads selected processes into main memory.
- Controls how many processes enter the system.
- Controls the degree of multiprogramming.

Short-term Scheduler (CPU Scheduler)

- Selects a process from the ready queue.
- Allocates the CPU to the selected process.
- Works very frequently and must make decisions quickly.

Medium-term Scheduler

- Temporarily removes processes from main memory (swap out).
- Can bring processes back later (swap in).
- Helps manage memory and the number of active processes.

Scheduler Diagram

diagram

Differences

table

Conclusion

Long-term selects jobs, short-term selects a process for the CPU, and medium-term manages swapping.

Remember: Long-term = Selects jobs; Short-term = Selects CPU process; Medium-term = Swapping.

```
JOB POOL
   |
   v
LONG-TERM SCHEDULER
   |
   v
MAIN MEMORY / READY QUEUE
   |
   v
SHORT-TERM SCHEDULER
   |
   v
  CPU

MEDIUM-TERM SCHEDULER
      <--> SWAP OUT / IN
```

Long-term	Short-term	Medium-term
Selects jobs from job pool.	Selects process from ready queue.	Temporarily removes/restores processes.
Loads processes into memory.	Allocates CPU.	Manages swapping.
Works less frequently.	Works very frequently.	Works when required.
Controls multiprogramming.	Controls CPU execution.	Helps manage memory.
Job Scheduler.	CPU Scheduler.	Swapper.

Q11. Explain CPU Scheduling Algorithms with examples (FCFS, SJF, Priority, Round Robin).

5 Marks – Expanded 10 Marks Format

1. Introduction

CPU Scheduling selects a process from the ready queue and assigns the CPU to it. An algorithm decides which process gets the CPU first.

FCFS

- First-Come, First-Served: process arriving first executes first.
- Follows FIFO and is non-preemptive.
- Example: people served in a queue.
- Advantage: simple. Disadvantage: Convoy Effect and potentially high waiting time.

SJF

- Shortest Job First selects the available process with the smallest CPU burst time.
- It can be preemptive or non-preemptive.
- Advantage: can give low average waiting time when burst times are known.
- Disadvantage: burst time is hard to predict; long processes may starve.

Priority Scheduling

- Each process has a priority; the highest-priority process is selected.
- It can be preemptive or non-preemptive.
- In this answer, smaller priority number means higher priority.
- Starvation can be reduced using aging.

Round Robin

- Each process gets a fixed Time Quantum.
- If it does not finish, it moves to the end of the ready queue.
- It is preemptive and suitable for time-sharing systems.
- A very small quantum causes more context switching; a very large quantum behaves like FCFS.

Comparison

table

Conclusion

FCFS uses arrival order, SJF uses burst time, Priority uses priority and Round Robin uses a fixed time quantum.

Remember: FCFS = Arrival; SJF = Shortest Job; Priority = Importance; RR = Time Quantum.

```
FCFS: P1 -> P2 -> P3 -> CPU
SJF:  shortest burst first
PRIORITY: highest priority first
ROUND ROBIN: each process gets a time slice
```

FCFS	SJF	Priority	Round Robin
Arrival order.	Shortest burst time.	Priority value.	Fixed time quantum.
Non-preemptive.	Preemptive or non-preemptive.	Preemptive or non-preemptive.	Preemptive.
Simple.	Can reduce average waiting time.	Important jobs first.	Each process gets a turn.
Convoy Effect.	Long jobs may starve.	Low-priority jobs may starve.	Context-switch overhead.

Q12. Explain the Context Switching mechanism, steps involved and overhead factors.

5 Marks – Expanded 10 Marks Format

1. Introduction

Context Switching is changing the CPU from one running process to another. The OS saves the current process information and loads the next process information. Example: switching CPU execution from a browser to a music player.

Process Context

- Program Counter: address of the next instruction.
- CPU Registers: temporary CPU values.
- Process State: current state of the process.
- This information is maintained in the PCB.

Steps

- An interrupt or scheduling event occurs.
- The OS saves the current process state in its PCB.
- The CPU scheduler selects another process.
- The OS restores the selected process state from its PCB.
- The CPU resumes execution of the selected process.

Context Switching Diagram

diagram

Example

- CPU executes P1 (browser).
- An interrupt occurs and the OS saves P1 information.
- The scheduler selects P2 (music player).
- The OS restores P2 information and executes it.
- P1 can continue later from where it stopped.

Overhead Factors

- Saving and restoring registers.
- Memory-management operations.
- Cache effects: cache contents from the previous process may be less useful.
- Time spent running the scheduler.
- Other kernel operations.

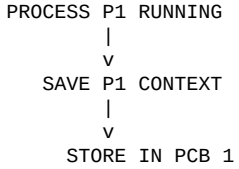
Advantages and Disadvantages

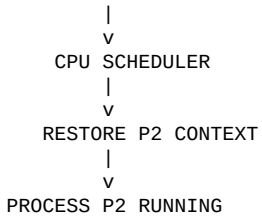
- Advantages: supports multitasking, lets processes share CPU time and allows processes to resume.
- Disadvantages: consumes CPU time, adds overhead and frequent switching may reduce performance.

Conclusion

Context switching allows the OS to switch CPU execution from one process to another by saving and restoring process states using PCBs.

Remember: Save Old Process → Select New Process → Restore New Process → Continue Execution.




|
v
CPU SCHEDULER
|
v
RESTORE P2 CONTEXT
|
v
PROCESS P2 RUNNING

UNIT 2 – CPU SCHEDULING NUMERICAL PROBLEMS

Four complete worked examples: FCFS, SJF, Priority and Round Robin. Each problem uses the same process data so the results can be compared.

Important Formulas

- Completion Time (CT): time at which a process finishes.
- Turnaround Time (TAT) = CT – AT.
- Waiting Time (WT) = TAT – BT.
- Average Waiting Time (AWT) = Total WT ÷ Number of Processes.
- Average Turnaround Time (ATAT) = Total TAT ÷ Number of Processes.

Common Given Data

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	8	2
P2	1	4	3
P3	2	9	1
P4	3	5	4

Assumptions: Priority 1 is the highest priority. SJF and Priority are non-preemptive. Round Robin uses Time Quantum = 3 units. Ties are handled in arrival order.

Problem 1. FCFS Scheduling

Step 1: Execution order

Arrival order is P1 → P2 → P3 → P4.

Step 2: Gantt Chart

```
P1       P2   P3         P4
|--------|----|---------|-----|
0        8    12        21    26
```

Step 3: Completion Time

P1 = 0+8=8; P2 = 8+4=12; P3 = 12+9=21; P4 = 21+5=26.

Step 4: Turnaround Time

TAT=CT–AT. P1=8–0=8; P2=12–1=11; P3=21–2=19; P4=26–3=23.

Step 5: Waiting Time

WT=TAT–BT. P1=8–8=0; P2=11–4=7; P3=19–9=10; P4=23–5=18.

Final Calculation Table

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	12	11	7
P3	2	9	21	19	10
P4	3	5	26	23	18

Average Waiting Time: AWT = (0+7+10+18)/4 = 35/4 = 8.75 units

Average Turnaround Time: ATAT = (8+11+19+23)/4 = 61/4 = 15.25 units

Problem 2. SJF – Non-Preemptive

Step 1: Execution order

At time 0 only P1 has arrived, so it must run first. At time 8, P2, P3 and P4 are ready. Their burst times are 4, 9 and 5. Therefore order is P1 → P2 → P4 → P3.

Step 2: Gantt Chart

```
P1       P2    P4     P3
|--------|----|-----|---------|
0        8    12    17        26
```

Step 3: Completion Time

P1=8; P2=8+4=12; P4=12+5=17; P3=17+9=26.

Step 4: Turnaround Time

TAT=CT–AT. P1=8; P2=12–1=11; P3=26–2=24; P4=17–3=14.

Step 5: Waiting Time

WT=TAT–BT. P1=0; P2=11–4=7; P3=24–9=15; P4=14–5=9.

Final Calculation Table

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	12	11	7
P3	2	9	26	24	15
P4	3	5	17	14	9

Average Waiting Time: AWT = (0+7+15+9)/4 = 31/4 = 7.75 units

Average Turnaround Time: ATAT = (8+11+24+14)/4 = 57/4 = 14.25 units

Problem 3. Priority Scheduling – Non-Preemptive

Step 1: Execution order

At time 0 only P1 has arrived, so it runs first. At time 8, all remaining processes have arrived. Priority 1 is highest: P3 (1), then P2 (3), then P4 (4). Order is P1 → P3 → P2 → P4.

Step 2: Gantt Chart

```
P1       P3          P2    P4
|--------|---------|----|-----|
0        8         17   21    26
```

Step 3: Completion Time

P1=8; P3=8+9=17; P2=17+4=21; P4=21+5=26.

Step 4: Turnaround Time

TAT=CT–AT. P1=8; P2=21–1=20; P3=17–2=15; P4=26–3=23.

Step 5: Waiting Time

WT=TAT–BT. P1=0; P2=20–4=16; P3=15–9=6; P4=23–5=18.

Final Calculation Table

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	4	21	20	16
P3	2	9	17	15	6
P4	3	5	26	23	18

Average Waiting Time: AWT = (0+16+6+18)/4 = 40/4 = 10 units

Average Turnaround Time: ATAT = (8+20+15+23)/4 = 66/4 = 16.5 units

Problem 4. Round Robin Scheduling

Step 1: Given

Time Quantum (TQ) = 3 units. Each process gets up to 3 units in one turn. If unfinished, it moves to the back of the ready queue.

Step 2: Execute each round

Round	Process	Execution interval	Remaining BT
1	P1	0–3	5
1	P2	3–6	1
1	P3	6–9	6
1	P4	9–12	2
2	P1	12–15	2
2	P2	15–16	0 (finished)
2	P3	16–19	3
2	P4	19–21	0 (finished)
3	P1	21–23	0 (finished)
3	P3	23–26	0 (finished)

Step 3: Gantt Chart

```
P1  P2  P3  P4  P1 P2  P3  P4 P1 P3
|---|---|---|---|---|--|---|--|--|---|
0   3   6   9  12  15 16  19 21 23  26
```

Step 4: Completion Time

P1 finishes at 23; P2 finishes at 16; P3 finishes at 26; P4 finishes at 21.

Step 5: Turnaround Time

TAT=CT–AT. P1=23–0=23; P2=16–1=15; P3=26–2=24; P4=21–3=18.

Step 6: Waiting Time

WT=TAT–BT. P1=23–8=15; P2=15–4=11; P3=24–9=15; P4=18–5=13.

Final Calculation Table

Process	AT	BT	CT	TAT	WT
P1	0	8	23	23	15
P2	1	4	16	15	11
P3	2	9	26	24	15
P4	3	5	21	18	13

Average Waiting Time: (15+11+15+13)/4 = 54/4 = 13.5 units.

Average Turnaround Time: (23+15+24+18)/4 = 80/4 = 20 units.

Final Comparison – All Four Algorithms

Algorithm	Average WT	Average TAT
FCFS	8.75	15.25
SJF	7.75	14.25
Priority	10.00	16.50
Round Robin	13.50	20.00

How to Identify the Algorithm in an Exam

Question clue	Algorithm
First arrival executes first.	FCFS
Smallest burst time executes first.	SJF
Highest priority executes first.	Priority Scheduling
Fixed time quantum is given.	Round Robin

Exam reminder: Always write the given data, identify the scheduling order, draw the Gantt chart, calculate CT, then TAT and WT, and finally calculate both averages.